

Solution 17 December, 2024

Course title: Network Optimization

Course no: 42115

Aids allowed: All

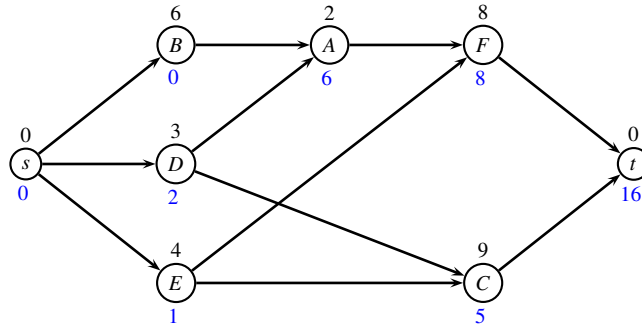
Exam duration: 4 hours

Critical path with time windows

Assume that each task j has a corresponding time windows $[e_j, \ell_j]$ denoting the earliest and latest time when the task may start.

activity i	description	duration d_i	predecessor	e_j	ℓ_j
A	Boil apples	2	B, D	0	∞
B	Peel apples and cut into cubes	6		0	∞
C	whip cream	9	D, E	0	6
D	slice vanilla bean, scrape out seeds	3		2	3
E	fry bread-crumbs with butter	4		1	∞
F	mix apples and bread-crumbs	8	A, E	0	∞

Answer 1 Introduce start and end nodes s, t . Add edges (s, i) to nodes i with no predecessor, and edges (i, t) to nodes i with no successor. Then we have the topological ordering



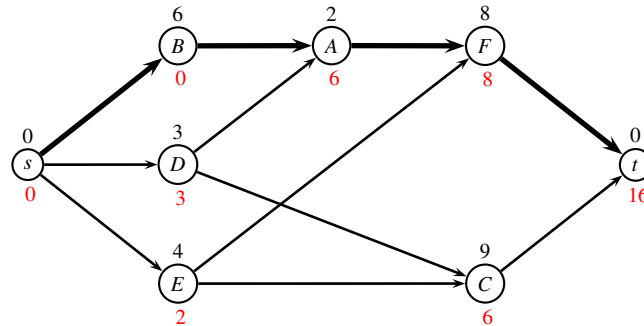
Let U_i be the earliest starting time of task i (marked with blue). *Forward procedure:* Consider the nodes j in topological order. Starting with $U_s = 0$, calculate the earliest time U_j each activity j can be started, using the recursion $U_j = \max \{e_j, \max_{(i,j) \in E} \{U_i + p_i\}\}$. The *makespan* is $T = U_t = 16$. ■

Note: An alternative method is to add predecessor tasks to handle e_j in time windows. For each task j with $e_j > 0$ add a dummy task between s and j with duration e_j .

Note: At this stage it is not possible to add successor tasks of duration ℓ_j after each node j .

Answer 2 Let V_i be the latest starting time (marked with red). *Backward procedure:* Consider the nodes j in backward topological order. Starting with $V_t = T$, calculate the latest time V_i each activity

i can be started using the recursion $V_i = \min \{ \ell_i, \min_{(i,j) \in E} \{ V_j - p_i \} \}$.



■

Note: An alternative method is to add successor tasks to handle ℓ_j in time windows. For each task j with $\ell_j < \infty$ add a dummy task between j and t with duration $T - \ell_j - p_j$.

Answer 3 The critical path is found where nodes have $U_i = V_i$. It is indicated with fat edges above, as $sBAFt$. ■

Note: When time windows are included in the recursion, the critical path is not guaranteed to be a full path from s to t (but it is in our case).

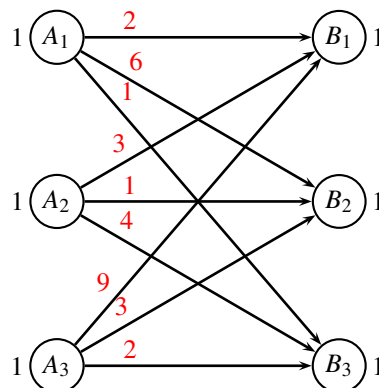
Assignment problem

In the *Assignment problem* we are asked to assign a number of resources A_i to an equal number of activities B_j , so as to minimize total cost of allocation. Each resource should be assigned to exactly one activity, and each activity should have exactly one resource assigned.

The below cost matrix $C = \{c_{ij}\}$ says how much it costs to assign resource i to activity j .

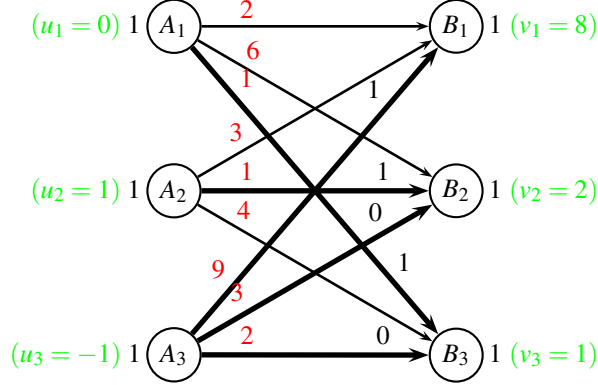
	B_1	B_2	B_3
A_1	2	6	1
A_2	3	1	4
A_3	9	3	2

Answer 4 Construct a bipartite graph with nodes A_i and B_j . All nodes A_i and B_j have a production/demand of 1.



■

Answer 5 We first consider A_1 , and send 1 unit to B_3 . Next, we consider A_2 , and send 1 unit to B_2 . Finally, we consider A_3 , and send 1 unit to B_1 . This gives the following greedy solution:



The cost of the solution is $z = 1 \cdot 1 + 1 \cdot 1 + 1 \cdot 9 = 11$.

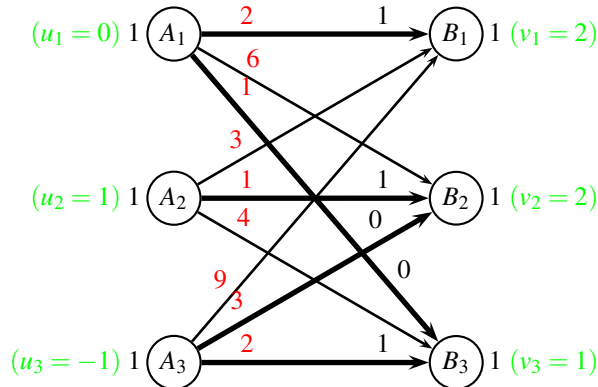
Fat edges on the graph indicate the basis. Since we have only 3 basis variables, we add two additional fat edge (A_3, B_2) . and (A_3, B_3) . (These can be arbitrarily chosen as long as the fat edges are a tree).

Dual values (indicated with green) are calculated by finding the shortest path from node A_1 , using only the fat edges. ■

Answer 6 The reduced costs are calculated as $\bar{c}_{ij} = c_{ij} + u_i - v_j$. This gives

$$\begin{aligned}\bar{c}_{11} &= 2 + 0 - 8 = -6 \\ \bar{c}_{12} &= 6 + 0 - 2 = 4 \\ \bar{c}_{21} &= 3 + 1 - 8 = -4 \\ \bar{c}_{23} &= 4 + 1 - 1 = 4\end{aligned}$$

The most negative reduced cost is for edge (A_1, B_1) giving the cycle $(A_1, B_1, A_3, B_3, A_1)$. Sending $\theta = \min\{1, 1\} = 1$ units along the cycle gives:



Edge A_1B_1 becomes fat while bottleneck edge A_3B_1 becomes thin. The cost of the solution is $z = 1 \cdot 2 + 1 \cdot 1 + 1 \cdot 2 = 5$.

The reduced costs are calculated as $\bar{c}_{ij} = c_{ij} + u_i - v_j$. This gives

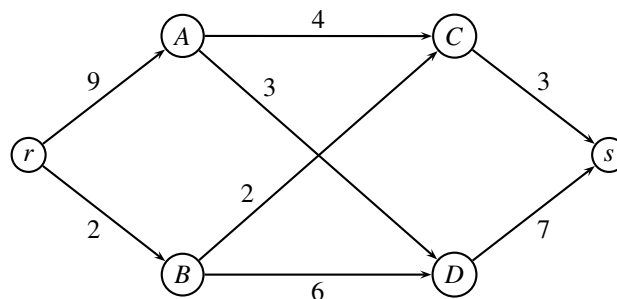
$$\begin{aligned}\bar{c}_{12} &= 6 + 0 - 2 = 4 \\ \bar{c}_{21} &= 3 + 1 - 2 = 2 \\ \bar{c}_{23} &= 4 + 1 - 1 = 4 \\ \bar{c}_{31} &= 9 + (-1) - 2 = 6\end{aligned}$$

No more negative reduced costs, so we finish. The optimal solution is $z = 5$. ■

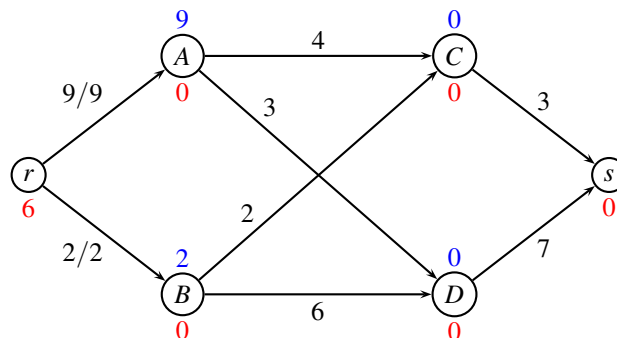
Note: We could also have chosen A_1B_3 as bottleneck edge, since they both have flow 1. In the next iteration we will find a cycle of capacity $\theta = 0$, giving us a new basis. Then we can show that all reduced costs are positive.

Maximum flow problem

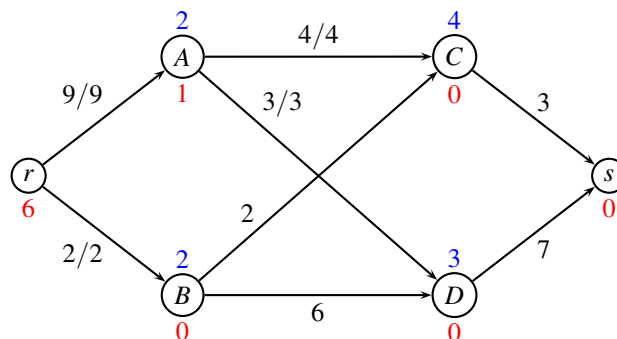
Consider the following maximum flow problem where the capacity u_{ij} is indicated on edge $(i, j) \in E$.



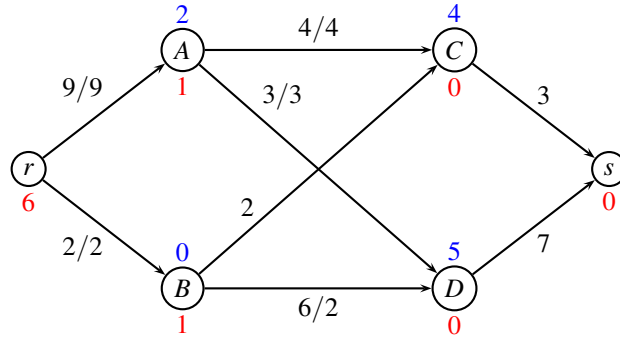
Answer 7 For node i , red value is label d_i , blue value is flow excess $f_x(i)$. Initialize: Set $d_r = n$ and all other $d_i = 0$. Push as much as possible from r .



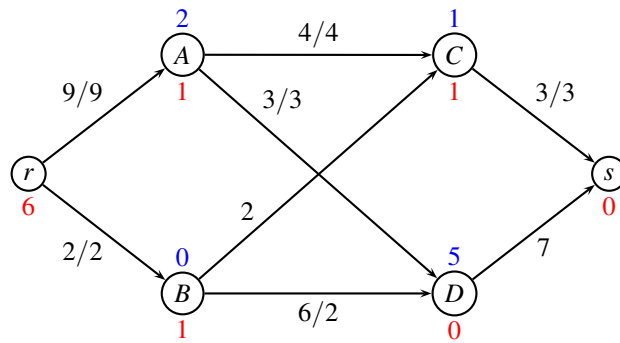
Iteration 1: relabel A to 1, push 4 on AC , and 3 on AD



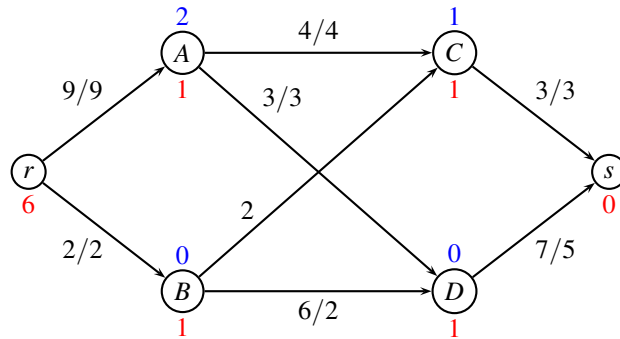
Iteration 2: relabel B to 1, push 2 on BD



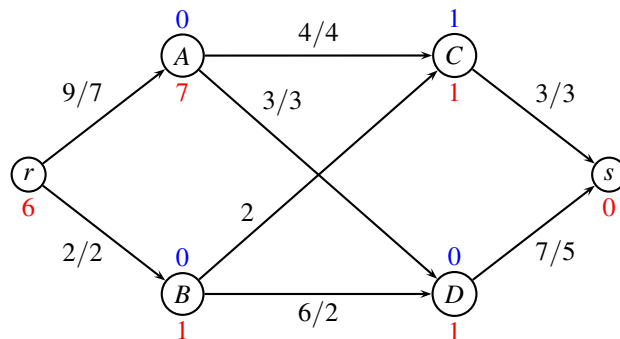
Iteration 3: relabel C to **1**, push 3 on Cs



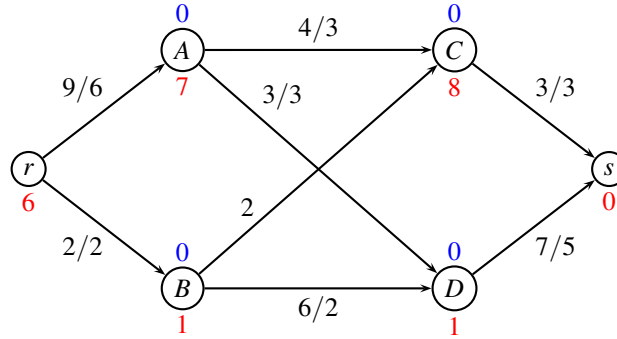
Iteration 4: relabel D to **1**, push 5 on Ds



Iteration 5: relabel A to **7** (the only outgoing edge in residual graph E_x is Ar), push 2 on Ar



Iteration 6: relabel C to **8** (the only outgoing edge in E_x is CA), push 1 on CA , push 1 on Ar

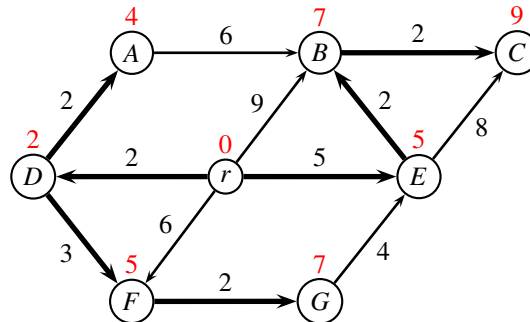


Flow is feasible, hence we stop. The optimal solution is $|f| = 6 + 2 = 8$. ■

Note: In iteration 5, if we relabel C instead of A , then we may end up with several iterations back and forth between A and C .

Answer 8 A minimum cut is found by selecting an unused label value, e.g. $k = 2$. Then we have $R = \{i \mid d_i > k\} = \{r, A, C\}$ and $\bar{R} = \{i \mid d_i < k\} = \{B, D, s\}$. The capacity of the cut is $u(R) = 2 + 3 + 3 = 8$. ■

A* algorithm for shortest path

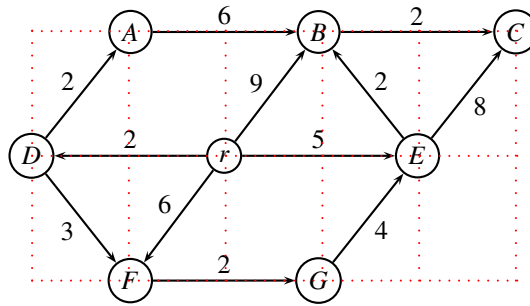


Answer 9 Running Dijkstra's algorithm. Each row indicates the node we are considering. We get the distances y_i as follows

node	r	A	B	C	D	E	F	G
init	0	∞	∞	∞	∞	∞	∞	∞
r			9		2	5	6	
D		4					5	
A								
F								7
E			7	13				
G								
B				9				
C								

For each node, the optimal distance is marked with bold in the above table. The distances are indicated at the above graph.

The shortest path tree is found by backtracking through the predecessors for each node. For instance, node C (distance 9) was updated when we considered node B , hence we mark edge BC fat. The resulting shortest path tree is shown above. ■



If we only are interested in solving the shortest distance to a specific terminal node $t = C$, we can use the A^* algorithm which makes use of heuristic distances to guide the search.

The Manhattan distance between two nodes (x_1, y_1) and (x_2, y_2) is measured as $h = |x_1 - x_2| + |y_1 - y_2|$. For instance, the Manhattan distance from r to C is $3 + 1 = 4$, while the distance from F to C is $4 + 2 = 6$.

Answer 10 The Manhattan distances are:

i	r	A	B	C	D	E	F	G
h_i	4	4	2	0	6	2	6	4

■

The following A^* algorithm finds the shortest path from r to t . It is identical to Dijkstra's algorithm, except that in every iteration we select the node i with lowest value of $y_i + h_i$. (The normal Dijkstra algorithm selects the node with lowest value of y_i). Furthermore, the algorithm stops when node t is added to T (i.e. marked "red").

```

1  $T \leftarrow \emptyset$ ;
2  $y_r \leftarrow 0$ ;  $y_i \leftarrow \infty$  for all other  $i$ ;
3 while  $V \setminus T \neq \emptyset$  do
4   select an  $i \in V \setminus T$  minimizing  $y_i + h_i$ ;
5   for  $\{(i, j) \in E : j \notin T\}$  do
6     if  $y_i + w_{ij} < y_j$  then
7        $y_j \leftarrow y_i + w_{ij}$ ;
8    $T \leftarrow T \cup \{i\}$ ;
9   if  $i = t$  then
10    stop

```

Answer 11 Running the A^* algorithm, we get the distances y_i and values $y_i + h_i$ as follows

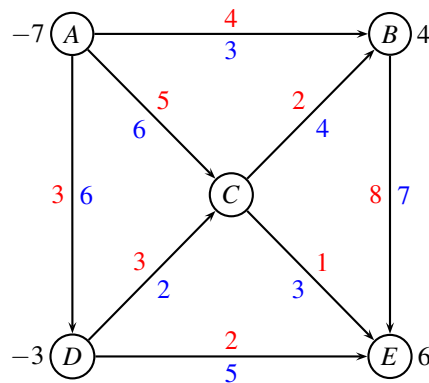
node	r	A	B	C	D	E	F	G
h_i	4	4	2	0	6	2	6	4
init	0	4	∞ ∞	∞ ∞	∞ ∞	∞ ∞	∞ ∞	∞ ∞
r			9 11		2 8	5 7	6 12	
E			7 9	13 13				
D		4 8					5 11	
A								
B				9 9				
C								

We stop as soon as $t = C$ is reached. As before, the shortest distance to node C is 9. The shortest path is found by following the predecessors (marked with boldface): The predecessor of C is B . The predecessor of B is E . The predecessor of E is r . Hence, the shortest path is $r \rightarrow E \rightarrow B \rightarrow C$, with cost $5 + 2 + 2 = 9$.

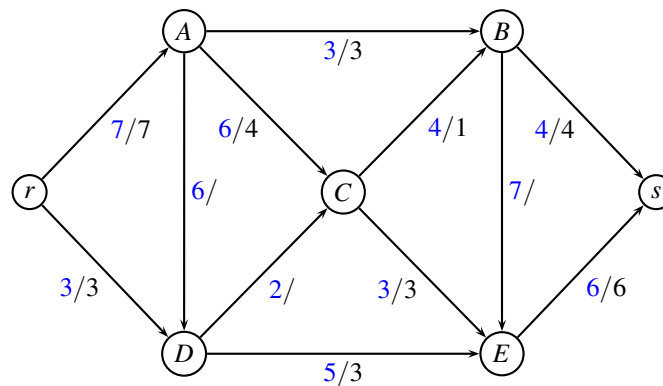
Compared to the previous question, fewer updates are needed. However, we do not have a shortest path tree (only the path $r \rightarrow C$), since we terminated early. ■

Minimum cost flow problem

Consider the following minimum cost flow problem costs w_{ij} are indicated with **red**, capacities u_{ij} are indicated with **blue**, and demands d_i are indicated in black.



Answer 12 We add a root node r and connect it to all nodes i having a negative demand d_i using an edge (r, i) with capacity $-d_i$. We add a sink node s and connect it to all nodes i having a positive demand d_i using an edge (i, s) with capacity d_i . All costs are removed. In this way we get the following maximum flow problem: In this way we get the following solution

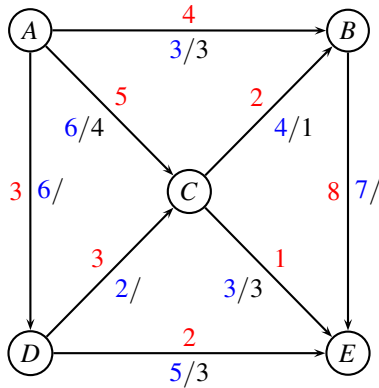


In order to solve the problem we use the following augmenting paths

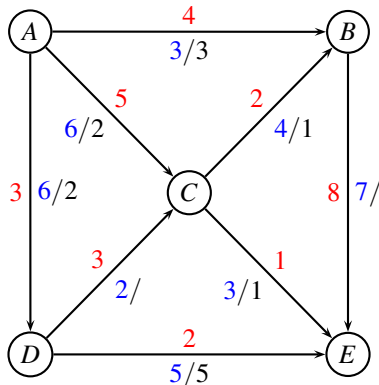
- send 3 units along path $rABs$
- send 1 units along path $rACBs$
- send 3 units along path $rACEs$
- send 3 units along path $rDEs$

This results in a solution of cost 43. ■

Answer 13 We use the **augmenting cycle** algorithm. We have the initial solution:

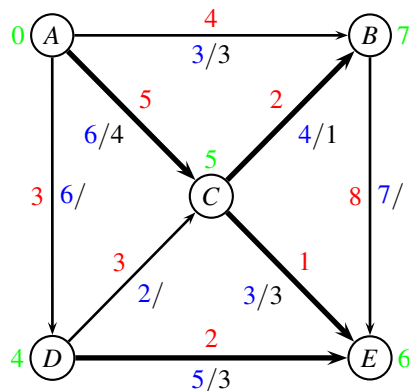


We find a negative cycle $ADECA$ with cost $3 + 2 - 1 - 5 = -1$. We can send $\theta = \min\{6, 2, 3, 4\} = 2$ units.



The cost of the solution is 41. No more negative cycles exist, so we terminate.

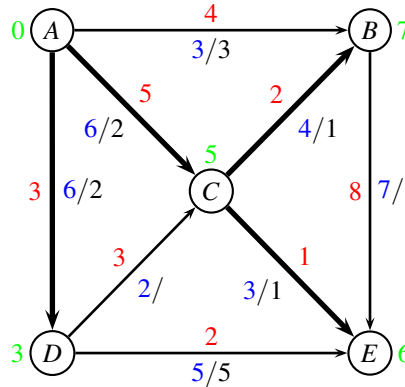
We can also use the **network simplex** algorithm. We have the initial solution:



The tree is $T = \{AC, CB, CE, DE\}$, lower bound edges are $L = \{AD, BE, DC\}$, upper bound edges are $U = \{AB\}$. Dual costs are calculated as distance from A (marked with green above). We find the reduced costs as $\bar{c}_{ij} = c_{ij} + u_i - u_j$, getting

$$\begin{aligned}\bar{c}_{AB} &= 6 + 0 - 7 = -1 && (U \text{ edge}) \\ \bar{c}_{AD} &= 3 + 0 - 4 = -1 && (L \text{ edge}) \\ \bar{c}_{BE} &= 8 + 7 - 6 = 9 && (L \text{ edge}) \\ \bar{c}_{DC} &= 3 + 4 - 5 = 2 && (L \text{ edge})\end{aligned}$$

Edge AD is lower bound edge, and has negative reduced cost. The corresponding cycle is $ADECA$ with bottleneck capacity $\theta = \min\{6, 2, 3, 4\} = 2$. Sending 2 units along the cycle. AD becomes fat, and bottleneck edge DE becomes thin.



The tree is $T = \{AC, AD, CB, CE\}$, lower bound edges are $L = \{BE, DC\}$, upper bound edges are $U = \{AB, DE\}$.

Dual costs are calculated as distance from A. We find the reduced costs:

$$\begin{aligned}\bar{c}_{AB} &= 6 + 0 - 7 = -1 & (U \text{ edge}) \\ \bar{c}_{BE} &= 8 + 7 - 6 = 9 & (L \text{ edge}) \\ \bar{c}_{DC} &= 3 + 4 - 5 = 2 & (L \text{ edge}) \\ \bar{c}_{DE} &= 2 + 4 - 6 = 0 & (L \text{ edge})\end{aligned}$$

No lower bound edges have negative reduced cost, and no upper bound edges have positive reduced cost. So we terminate. The cost of the solution is 41. ■

Note: Depending on the start solution, it may be necessary with more iterations. If you find an optimal solution as start solution, try to modify it, so you at least have one iteration.

```

minimize
  4 xAB + 5 xAC + 3 xAD + 8 xBE + 2 xCB + 1 xCE + 3 xDC + 2 xDE
subject to
A: -xAB - xAC - xAD          = -7
B:  xAB + xCB - xBE          = 4
C:  xAC + xDC - xCB - xCE    = 0
D:  xAD - xDC - xDE          = -3
E:  xBE + xCE + xDE          = 6

bounds
  xAB <= 3
  xAC <= 6
  xAD <= 6
  xBE <= 7
  xCB <= 4
  xCE <= 3
  xDC <= 2
  xDE <= 5
end

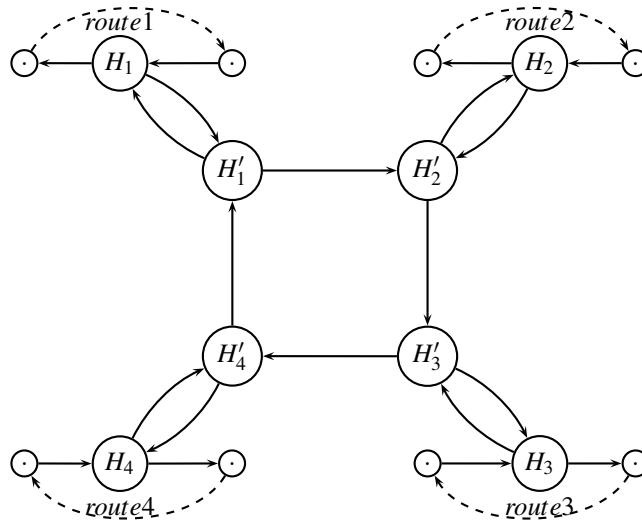
```

```

CPLEX> opt
Dual simplex - Optimal: Objective = 4.1000000000e+01
CPLEX> dis sol var -
Variable Name          Solution Value
xAB                     3.000000
xAC                     2.000000
xAD                     2.000000
xCB                     1.000000
xCE                     1.000000
xDE                     5.000000
All other variables in the range 1-8 are 0.

```

Modeling a hub



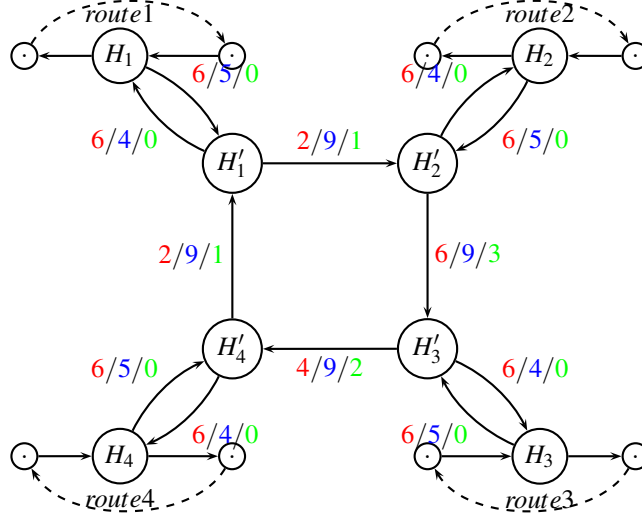
We study how the hub port H in a liner shipping network, HUBLINE, can be formulated in graph terms as part of a multi-commodity flow network. We have four routes arriving to the hub as follows:

route	1	2	3	4
weekday	Monday	Tuesday	Friday	Sunday

We assume that each vessel is only one day in the port, and it is assumed that all loading and unloading is done during this day.

Crane capacity: The crane capacity is $u = 5$ for unloading and $u = 4$ for loading a vessel. *Transshipment cost:* The cost of loading/unloading is 6 euro per container. *Transshipment time:* The time for transferring a container from one vessel to another can be calculated from the time table. *Storage capacity:* The port has a limited storage capacity of $\bar{u} = 9$ containers on shore. *Storage cost:* The shipping company must pay $\bar{c} = 2$ euro for each container each night it is stored in the port.

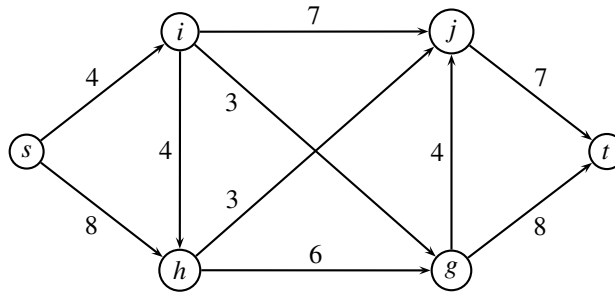
Answer 14 All values can be modeled as follows:



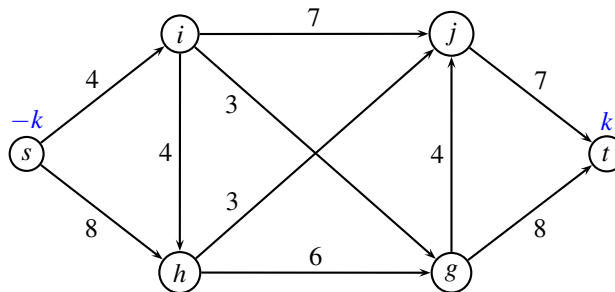
■

Maximum flow with lower bound on edges

Given a directed graph $G = (V, E, u)$, root/sink (s, t) and a target for the flow k . We assume that all capacities u are positive integers.

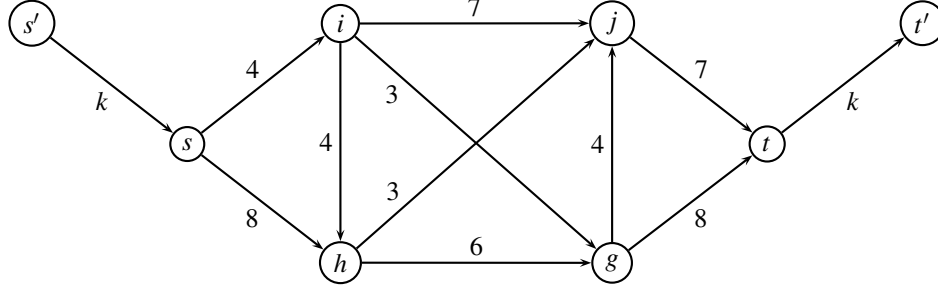


Answer 15 We formulate it as a circulation problem, with demand $d_s = -k$ and $d_t = k$.



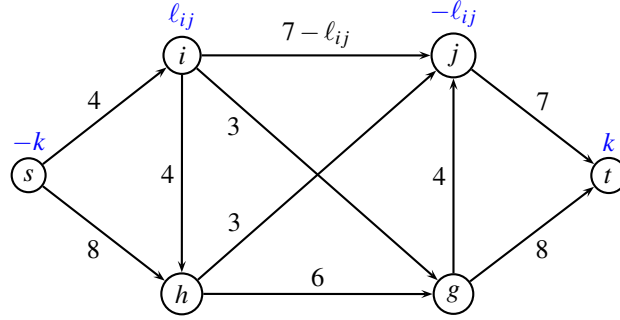
To modify the circulation problem to a normal maximum-flow problem a super-source s' and super-sink t' is added. Furthermore we add edge (s', s) with capacity k , and edge (t, t') with capacity k . If we can find a flow of value $|f| = k$ (i.e. all edges from s' are saturated), the circulation problem is

feasible.



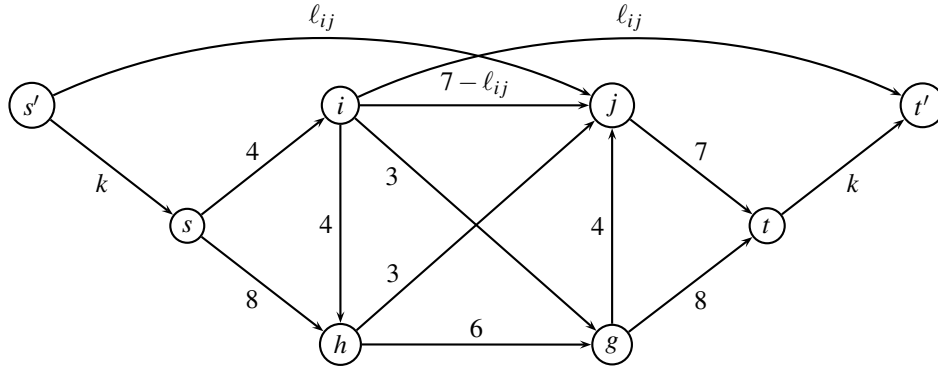
■

Answer 16 We formulate it as a circulation problem, where node i gets demand ℓ_{ij} and node j gets demand $-\ell_{ij}$. Furthermore, we have demand $d_s = -k$ and $d_t = k$.



To modify the circulation problem to a normal maximum-flow problem a super-source s' and super-sink t' is added. We add edge (s', s) with capacity k and (s', j) with capacity ℓ_{ij} . We add edge (t, t') with capacity k and (i, t') with capacity ℓ_{ij} . If we can find a flow of value $|f| = k + \ell_{ij}$ (i.e. all edges from s' are saturated), the circulation problem is feasible.

Notice that the method works for multiple edges with lower bound, by repeating the transformation for each edge.



■

Answer 17 We first find an upper bound U on the maximum flow $|f|$ as sum of the capacities for edges leaving s , or entering t .

$$U = \min \left\{ \sum_{(s,i) \in E} u_{si}, \sum_{(i,t) \in E} u_{it} \right\}$$

Now, use binary search to find a maximum flow. I.e. first choose $k = \lfloor U/2 \rfloor$, and solve the above circulation problem. If the problem is feasible we know that z is between k and U . Otherwise z is between 0 and $k - 1$. Repeat until the optimal value z^* has been found. This means that we will solve $\log U$ maximum-flow problems, which is a polynomial algorithm. ■